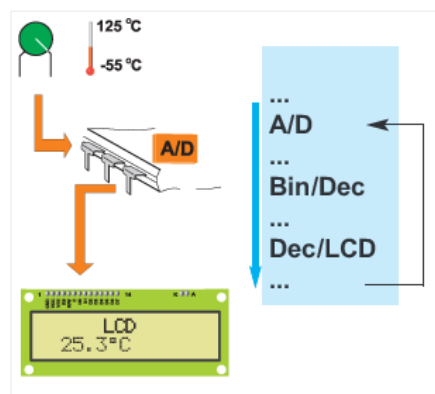


2.2 THE BASICS OF C PROGRAMMING LANGUAGE

MIKROELEKTRONIKA

The main idea of writing program in C language is to break a bigger problem down into several smaller pieces. Suppose it is necessary to write a program for the microcontroller that is going to measure temperature and show results on an LCD display. The process of measuring is performed by a sensor that converts temperature into voltage. The microcontroller uses its A/D converter to convert this voltage (analogue value) to a number (digital value) which is then sent to the LCD display via several conductors. Accordingly, the program is divided in four parts that you have to go through as per the following order:

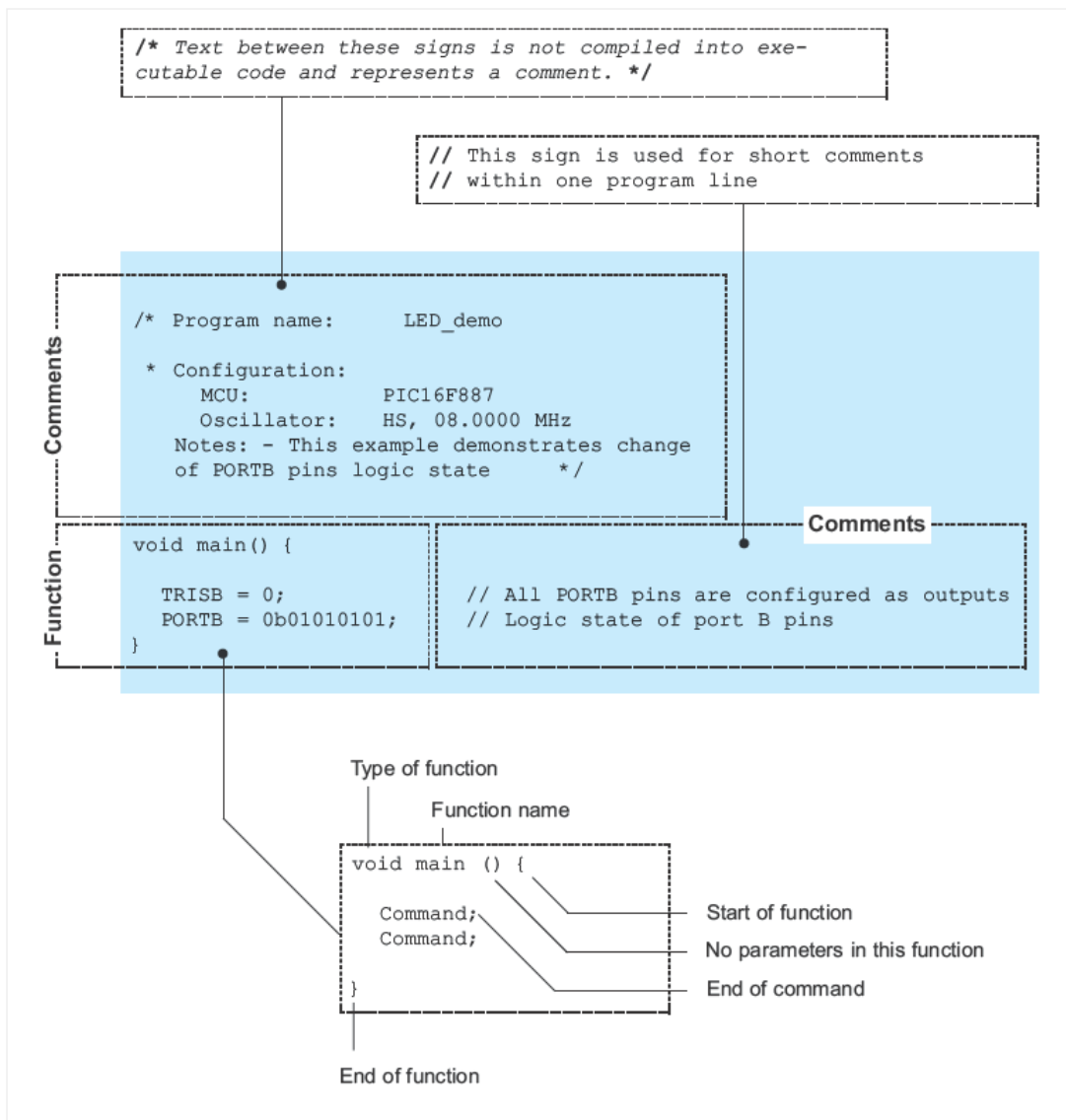


1. Activate and set built-in A/D converter;
2. Measure analogue value;
3. Calculate temperature; and
4. Send data in the proper form to LCD display.

As seen, the higher programming languages such as C enable you to solve this problem easily by writing four functions to be executed cyclically and over and over again.

*This book describes a very concrete application of C programming language, i.e. C language used for the **mikroC PRO for PIC** compiler. In this case, the compiler is used for programming PIC microcontrollers. Anyway, this note refers to details on the programming language that are intentionally left out herein because they have no practical application, rather than to variations on the standard C language (basically, there are no differences).*

Figure below illustrates the structure of a simple program, pointing out the parts it consists of.



COMMENTS

Comments are part of the program used to clarify the operation of the program or provide more information about it. Comments are ignored and not compiled into executable code by the compiler. Simply put, the compiler can recognize special characters used to designate where comments start and terminate and completely ignores the text inbetween during compilation. There are two types of such characters. One designates long comments extending several program lines, while the other designates short comments taking up a single line. Even though comments cannot affect the program execution, they are as important as any other part of the program, and here is why... A written program can always be improved, modified, upgraded, simplified... It is almost always done. Without comments, trying to understand even the simplest programs is waste of time.

DATA TYPES IN C LANGUAGE

There are several types of data that can be used in C programming language. A table below shows the range of values which these data can have when used in their basic form.

DATA TYPE	DESCRIPTION	SIZE (NUMBER OF BITS)	RANGE OF VALUES
char	Character	8	0 to 255
int	Integer	16	-32768 to 32767
float	Floating point	32	$\pm 1.17549435082 \cdot 10^{-38}$ to $\pm 6.80564774407 \cdot 10^{38}$
double	Double precision floating point	32	from $\pm 1.17549435082 \cdot 10^{-38}$ to $\pm 6.80564774407 \cdot 10^{38}$

By adding prefix (qualificator) to any data type, the range of its possible values changes as well as the number of memory bytes needed.

DATA TYPE	DATA TYPE WITH PREFIX	SIZE (NUMBER OF BITS)	RANGE
char	signed char	8	-128 to 128
int	unsigned int	16	0 to 65535
	short int	8	0 to 255
	signed short int	8	-128 to 127
	long int	32	0 to 4294967295
	signed long int	32	-2147483648 to 2147483647

VARIABLES

Any number changing its value during program operation is called a variable. Simply put, if the program adds two numbers (number1 and number2), it is necessary to have a value to represent what we in everyday life call the sum. In this case number1, number2 and sum are variables.

Declaring Variables

- Variable name can include any of the alphabetical characters A-Z (a-z), the digits 0-9 and the underscore character '_'. The compiler is case sensitive and differentiates between capital and small letters. Function and variable names usually contain lower case characters, while constant names contain uppercase characters.
- Variable names must not start with a digit.
- Some of the names cannot be used as variable names as already being used by the compiler itself. Such names are called the key words. The mikroC compiler recognizes in total of 33 such words:

MIKROC – KEYWORDS				
absolute	data	if	return	typedef
asm	default	inline	rx	typeid
at	delete	int	sfr	typename
auto	do	io	short	union
bit	double	long	signed	unsigned
bool	else	mutable	sizeof	using
break	enum	namespace	static	virtual
case	explicit	operator	struct	void
catch	extern	org	switch	volatile
char	false	pascal	template	while
class	float	private	this	
code	for	protected	throw	
const	friend	public	true	
continue	goto	register	try	

Pointers

A pointer is a special type of variable holding the address of character variables. In other words, the pointer 'points to' another variable. It is declared as follows:

```
1 type_of_variable *pointer_name;
```

In order to assign the address of a variable to a pointer, it is necessary to use the '=' character and write variable name preceded by the '&' character. In the following example, the pointer 'multiplex' is declared and assigned the

address of the first out of eight LED displays:

```
1 unsigned int *multiplex; // Declare name and type of pointer multiplex
2 multiplex = &display1;  // Pointer multiplex is assigned the address of
3                          // variable display1
```

To change the value of the pointed variable, it is sufficient to write the '*' character in front of its pointer and assign it a new value.

```
1 *multiplex = 6; // Variable display1 is assigned the number 6
```

Similarly, in order to read the value of the pointed variable, it is sufficient to write:

```
1 temp = *multiplex; // The value of variable display1 is copied to temp
```

Changing individual bits

There are a few ways to change only one bit of a variable. The simplest one is to specify the register name, bit's position or a name and desired state:

```
1 (PORTD.F3 = 0); // Clear the RD3 bit
2 ...
3 (PORTC.RELAY = 1); // Set the PORTC output bit (previously named RELAY)
4                  // RELAY must be defined as constant
```

Declarations

Every variable must be declared prior to being used for the first time in the program. Since variables are stored in RAM memory, it is necessary to reserve space for them (one, two or more bytes). You know what type of data you write or expect as a result of an operation, while the compiler does not know that. Don't forget, the program deals with variables to which you assigned the names *gate*, *sum*, *minimum* etc. The compiler recognizes them as registers of RAM memory. Variable types are usually assigned at the beginning of the program.

```
1 unsigned int gate1; // Declare name and type of variable gate1
```

Apart from the name and type, variables are usually assigned initial values at the beginning of the program as well. It is not a 'must-do' step, but a matter of good habits. In this case, it looks as follows:

```
1 unsigned int gate1; // Declare type and name of the variable
2 signed int start, sum; // Declare type and name of other two variables
3 gate1 = 20; // Assign variable gate1 an initial value
```

The process of assigning initial value and declaring type can be performed in one step:

```
1 unsigned int gate1=20; // Declare type, name and value of variable
```

If there are several variables being assigned the same initial value, the process can be even simplified:

```
1 unsigned int gate1=gate2=gate3=20;
2 signed int start=sm=0;
```

- Type of variable is not accompanied by the '+' or '-' sign by default. For example, char can be written instead of signed char (variable is a signed byte). In this case the compiler considers variable positive values.
- If you, by any chance, forget to declare variable type, the compiler will automatically consider it a signed integer. It means that such a variable will occupy two memory bytes and have values in the range of -32768 to +32767.

CONSTANTS

A constant is a number or a character having fixed value that cannot be changed during program execution. Unlike variables, constants are stored in the flash program memory of the microcontroller for the purpose of

saving valuable space of RAM. The compiler recognizes them by their name and prefix *const*.

INTEGER CONSTANTS

Integer constants can be decimal, hexadecimal, octal or binary. The compiler recognizes their format on the basis of the prefix added. If the number has no prefix, it is considered decimal by default. The type of a constant is automatically recognized by its size. In the following example, the constant MINIMUM will be automatically considered a signed integer and stored within two bytes of Flash memory (16 bits):

FORMAT	PREFIX	EXAMPLE
Decimal		const MAX = 100
Hexadecimal	0x or 0X	const MAX = 0xFF
Octal	0	const MAX = 016
Binary	0b or 0B	const MAX = 0b11011101

```
1 const MINIMUM = -100; // Declare constant MINIMUM
```

FLOATING POINT CONSTANTS

Floating point constants consist of an integer part, a dot, a fractional part and an optional e or E followed by a signed integer exponent.

```
1 const T_MAX = 32.60; // Declare temperature T_MAX
2 const T_MAX = 3.260E1; // Declare the same constant T_MAX
```

In both examples, a constant named T_MAX is declared to have the fractional value 32.60. It enables the program to compare the measured temperature to the meaningful constant instead of numbers representing it.

CHARACTER CONSTANTS (ASCII CHARACTERS)

A character constant is a character enclosed within single quotation marks. In the following example, a constant named I_CLASS is declared as A character, while a constant named II_CLASS is declared as B character.

```
1 const I_CLASS = 'A'; // Declare constant I_CLASS
2 const II_CLASS = 'B'; // Declare constant II_CLASS
```

When defined this way, the execution of the commands sending the I_CLASS and II_CLASS constants to an LCD display, will cause the characters A and B to be displayed, respectively.

STRING CONSTANTS

A constant consisting of a sequence of characters is called a string. String constants are enclosed within double quotation marks.

```
1 const Message_1 = "Press the START button"; // Message 1 for LCD
2 const Message_2 = "Press the RIGHT button"; // Message 2 for LCD
3 const Message_3 = "Press the LEFT button"; // Message 3 for LCD
```

In this example, sending the Message_1 constant to an LCD display will cause the message *'press the START button'* to be displayed.

ENUMERATED CONSTANTS

Enumerated constants are a special type of integer constants which make a program more comprehensive and easier to follow by assigning elements the ordinal numbers. In the following example, the first element in curly brackets is automatically assigned the value 0, the second one is assigned the value 1, the third one the value 2

etc.

```
1 enum MOTORS {UP, DOWN, LEFT, RIGHT}; // Declare constant MOTORS
```

On every occurrence of the words 'LEFT', 'RIGHT', 'UP' and 'DOWN' in the program, the compiler will replace them by the appropriate numbers (0-3). Concretely, if the port B pins 0, 1, 2 and 3 are connected to motors which make something goes up, down, left and right, the command for running motor 'RIGHT' connected to bit 3 of port B looks as follows:

```
1 PORTB.RIGHT = 1; // set the PORTB bit 3 connected to the motor 'RIGHT'
```

OPERATORS, OPERATIONS AND EXPRESSIONS

An operator is a symbol denoting particular arithmetic, logic or some other operation. There are more than 40 operations available in C language, but at most 10-15 of them are used in practice. Every operation is performed upon one or more operands which can be variables or constants. Besides, every operation features priority execution and associativity as well.

ARITHMETIC OPERATORS

Arithmetic operators are used in arithmetic operations and always return positive results. Unlike unary operations being performed upon one operand, binary operations are performed upon two operands. In other words, two numbers are required to execute a binary operation. For example: a+b or a/b.

OPERATOR	OPERATION
+	Addition
–	Subtraction
*	Multiplication
/	Division
%	Reminder

ASSIGNMENT OPERATORS

There are two types of assignments in C language:

- Simple operators assign values to variables using the common '=' character. For example: a = 8
- Compound assignments are specific to C language and consist of two characters as shown in the table. An expression can be written in a different way as well, but this one provides more efficient machine code.

OPERATOR	EXAMPLE	
	Expression	Equivalent
+=	a += 8	a = a + 8
-=	a -= 8	a = a – 8
*=	a *= 8	a = a * 8
/=	a /= 8	a = a / 8
%=	a %= 8	a = a % 8

INCREMENT AND DECREMENT OPERATORS

Increment and decrement by 1 operations are denoted by '++' and '--'. These characters can either precede or follow a variable. In the first case (++x), the x variable will be first incremented by 1, then used in expression. Otherwise, the variable will be first used in expression, then incremented by 1. The same applies to the decrement operation.

OPERATOR	EXAMPLE	DESCRIPTION
++	++a	Variable "a" is incremented by 1
	a++	
--	--b	Variable "b" is decremented by 1
	b--	

RELATIONAL OPERATORS

Relational operators are used in comparisons for the purpose of comparing two variables which can be integers (int) or floating point numbers (float). If an expression evaluates to true, a 1 is returned. Otherwise, a 0 is returned. This is used in expressions such as 'if the expression is true then...'

OPERATOR	MEANING	EXAMPLE	TRUTH CONDITION
>	is greater than	b > a	if b is greater than a
>=	is greater than or equal to	a >= 5	If a is greater than or equal to 5
<	is less than	a < b	if a is less than b
<=	is less than or equal to	a <= b	if a is less than or equal to b
==	is equal to	a == 6	if a is equal to 6
!=	is not equal to	a != b	if a is not equal to b

LOGIC OPERATORS

There are three types of logic operations in C language: logic AND, logic OR and negation (NOT). For the sake of clearness, logic states in tables below are represented as logic zero (0=false) and logic one (1=true). Logic operators return true (logic 1) if the expression evaluates to non-zero, and false (logic 0) if the expression evaluates to zero. This is very important because logic operations are commonly used upon expressions, not upon single variables (numbers) in the program. Therefore, logic operations refer to the truth of the whole expression.

For example: `1 && 0` is the same as (true expression) && (false expression)

The result is 0, i.e. – *False* in either case.

OPERATOR	LOGICAL AND		
&&	Operand1	Operand2	Result
	0	0	0
	0	1	0
	1	0	0
	1	1	1

OPERATOR	LOGICAL OR		
	Operand1	Operand2	Result
	0	0	0
	0	1	1
	1	0	1
	1	1	1

OPERATOR	LOGICAL NOT	
!	Operand1	Result
	0	1

1

0

BITWISE OPERATORS

Unlike logic operations being performed upon variables, the bitwise operations are performed upon single bits within operands. Bitwise operators are used to modify the bits of a variable. They are listed in the table below:

OPERAND	MEANING	EXAMPLE	RESULT
~	Bitwise complement	a = ~b	b = 5 a = -5
<<	Shift left	a = b << 2	b = 11110011 a = 11001100
>>	Shift right	a = b >> 2	b = 11110011 a = 00011110
&	Bitwise AND	c = a & b	a = 11100011 b = 11001100 c = 11000000
	Bitwise OR	c = a b	a = 11100011 b = 11001100 c = 11101111
^	Bitwise EXOR	c = a ^ b	a = 11100011 b = 11001100 c = 00101111

HOW TO USE OPERATORS?

- Except for assignment operators, two operators must not be written next to each other.

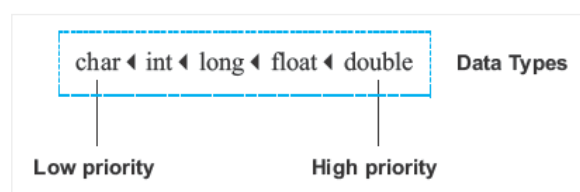
```
1 x**12; // such expression will generate an error
```

- Operators are grouped together using parentheses similar to arithmetic expressions. The expressions enclosed within parentheses are calculated first. If necessary, multiple (nested) parentheses can be used.
- Each operator has its priority and associativity as shown in the table.

PRIORITY	OPERATORS	ASSOCIATIVITY
High	() [] -> .	from left to right
	! ~ ++ -- +(unary) -(unary) *Pointer &Pointer	from right to left
	* / %	from left to right
	+ -	from left to right
	< >	from left to right
	< <= > >=	from left to right
	== !=	from left to right
	&	from left to right
	^	from left to right
		from left to right
	&&	from left to right
		from right to left
	?:	from right to left
Low	= += -= *= /= &= ^= = <= >=	from left to right

DATA TYPE CONVERSION

The main data types are put in hierarchical order as follows:



If two operands of different type are used in an arithmetic operation, the lower priority operand type is automatically converted into the higher priority operand type. In expressions free from assignment operation, the result is obtained in the following way:

- If the highest priority operand is of type *double*, then types of all other operands in the expression as well as the result are automatically converted into type *double*.
- If the highest priority operand is of type *long*, then types of all other operands in the expression as well as the result are automatically converted into type *long*.
- If the operands are of *long* or *char* type, then types of all other operands in the expression as well as the result are automatically converted into type *int*.

Auto conversion is also performed in assignment operations. The result of the expression right from the assignment operator is always converted into the type of variable left from the operator. If the result is of higher-ranked type, it is truncated or rounded in order to match the type of variable. When converting real data into integer, numbers following the decimal point are always truncated.

```
1 int x;      // Variable x is declared as integer int
2 x = 3;      // Variable x is assigned value
3 x += 3.14;  // Number PI (3.14) is added to variable x by performing
4             // the assignment operation
5
6 /* The result of addition is 6 instead of expected 6.14. To obtain the
7    expected result without truncating the numbers following the decimal
8    point, common addition should be performed (x+3.14), . */
```

CONDITIONAL OPERATORS

A condition is a common ingredient of the program. When met, it is necessary to perform one out of several operations. In other words '*If the condition is met (...), do (...). Otherwise, if the condition is not met, do (...)*'. Conditional operands **if-else** and **switch** are used in conditional operations.

CONDITIONAL OPERATOR if-else

The conditional operator can appear in two forms – as **if** and **if-else** operator.

Here is an example of the **if** operator:

```
1 if(expression) operation;
```

If the result of *expression* enclosed within brackets is not 0 (true), the *operation* is performed and the program proceeds with execution. If the result of *expression* is 0 (false), the *operation* is not performed and the program immediately proceeds with execution.

As mentioned, the other form combines both **if** and **else** operators:

```
1 if(expression) operation1 else operation2;
```

If the result of *expression* is not 0 (true), *operation1* is performed, otherwise *operation2* is performed. After performing either operation, the program proceeds with execution.

The syntax of the **if-else** statement is:

```
1 if(expression)
2 operation1
3 else
4 operation2
```

If either *operation1* or *operation2* is compound, a group of operations these consist of must be enclosed within curly brackets. For example:

```
1 if(expression) {
2 ... //
3 ... // operation1
4 ...} //
```

```
5 else
6 operation2
```

The **if-else** operator can be written using the conditional operator '?' as in example below:

```
1 (expression1)? expression2 : expression3
```

If *expression1* is not 0 (true), the result of the whole expression will be equal to the result obtained from *expression2*. Otherwise, if *expression1* is 0 (false), the result of the whole expression will be equal to the result obtained from *expression3*.

```
1 maximum = (a > b)? a : b // Variable maximum is assigned the value of
2                       // larger variable (a or b)
```

Switch OPERATION

Unlike the **if-else** statement which makes selection between two options in the program, the **switch** operator enables you to choose between several operations. The syntax of the **switch** statement is:

```
1 switch (selector)           // Selector is of char or int type
2 {
3     case constant1:
4         operation1           // Group of operators are executed if
5         ...                  // selector and constant1 are equal
6         break;
7     case constant2:
8         operation2           // Group of operators are executed if
9         ...                  // selector and constant2 are equal
10        break;
11        ...
12    default:
13        expected_operation    // Group of operators are executed if no
14        ...                  // constant is equal to selector
15        break;
16 }
```

The **switch** operation is executed in the following way: *selector* is executed first and compared to *constant1*. If match is found, statements in that case block are executed until the *break* keyword or the end of the **switch** operation is encountered. If no match is found, *selector* is further compared to *constant2* and if match is found, statements in that case block are executed until the *break* keyword is encountered and so on. If the selector doesn't match any constant, operations following the **default** operator are to be executed.

It is also possible to compare an expression with a group of constants. If it matches any of them, the appropriate operations will be executed:

```
1 switch (number) // number represents one day in a week. It is
2                 // necessary to determine whether it is a week-
3 {               // day or not.
4     case1:case2:case3:case4:case5: LCD_message = 'Weekday'; break;
5     case6:case7: LCD_message = 'Weekend'; break;
6     default:
7     LCD_message_1 = 'Choose one day in a week'; break;
8 }
```

PROGRAM LOOP

It is often necessary to repeat a certain operation for a couple of times in the program. A set of commands being repeated is called the program loop. How many times it will be executed, i.e. how long the program will stay in the loop, depends on the conditions to leave the loop.

While LOOP

The **while** loop looks as follows:

```
1 while(expression){
2     commands
3     ...
4 }
```

The *commands* are executed repeatedly (the program remains in the loop) until the *expression* becomes false. If the *expression* is false on entry to the loop, then the loop will not be executed and the program will proceed from the end of the **while** loop.

A special type of program loop is the *endless loop*. It is formed if the condition remains unchanged within the loop. The execution is simple in this case as the result in brackets is always true (1=1), which means that the program remains in the same loop:

```
1 while(1){
2     ... // Expressions enclosed within curly brackets will be
3     ... // endlessly executed (endless loop).
4 }
```

For LOOP

The **for** loop looks as follows:

```
1 for(initial_expression; condition_expression; change_expression) {
2     operations
3     ...
4 }
```

The execution of such program sequence is similar to the **while** loop, except that in this case the process of setting initial value (initialization) is performed within declaration. The *initial_expression* sets the initial variable of the loop, which is further compared to the *condition_expression* before entering the loop. *Operations* within the loop are executed repeatedly and after each iteration the value of expression is changed. The iteration continues until the *condition_expression* becomes false.

```
1 for(k=1; k<=5; k++) // Increase variable k 5 times (from 1 to 5) and
2     operation       // repeat expression operation every time
3     ...
```

Operation is to be performed five times. After that, it will be validated by checking that the expression $k \leq 5$ is false (after 5 iterations $k=5$) and the program will exit the **for** loop.

Do-while LOOP

The *do-while* loop looks as follows:

```
1 do
2 operation
3 while (check_condition);
```

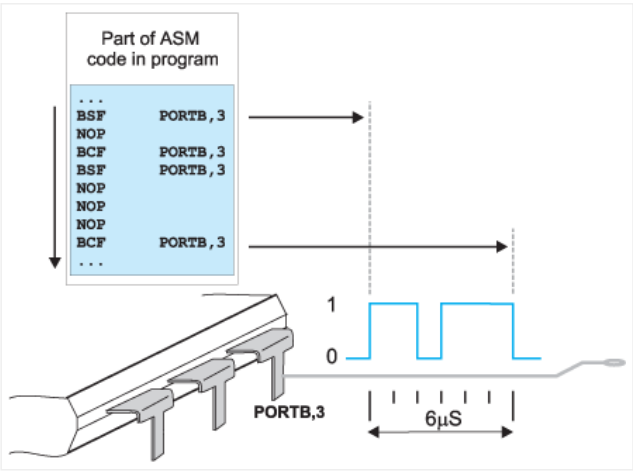
In this case, the *operation* is executed at least once regardless of whether the condition is true or false as the expression *check_condition* is executed at the end of the loop. If the result is not 0 (true), the procedure is repeated. In the following example, the program remains in **do-while** loop until the variable *a* reaches 1E06 (a million iterations).

```
1 a = 0; // Set initial value
2
3 do
4
5 a = a+1 // Operation in progress
6
7 while (a <= 1E06); // Check condition
```

WRITING CODE IN ASSEMBLY LANGUAGE

Sometimes the process of writing a program in C language requires parts of the code to be written in assembly language. This enables complicated parts of the program to be executed in a precisely defined way for exact period of time. For example, when it is necessary to have very short pulses (a few microseconds) appearing periodically on a microcontroller pin. In such and similar cases, the simplest solution is to use assembly code for

the part of the program controlling pulse duration.



One or more assembly instructions are inserted in the program written in C language using the **asm** command:

```
1 asm
2 {
3   Assembly language instructions
4   ...
5 }
```

Codes written in assembly language can use constants and variables previously defined in C language. Of course, as the whole program is written in C language, the rules thereof are applied when declaring these constants and variables.

```
1 unsigned char maximum = 100; // Declare variables: maximum = 100
2 asm
3 { // Start of assembly code
4   MOVF maximum,W // W = maximum = 100
5   ...
6 } // End of assembly code
```

ARRAYS

A group of variables of the same type is called an array. Elements of an array are called components, while their type is called the main type. An array is declared by specifying its name, type and the number of elements it will comprise:

```
1 component_type array_name [number_of_components];
```

Such a complicated definition for something so simple, isn't it? An array can be thought of as a shorter or longer list of variables of the same type where each of these is assigned an ordinal number (numbering always starts at zero). Such an array is often called a vector. The figure below shows an array named *shelf* which consists of 100 elements.

ARRAY "SHELF"	ELEMENTS OF ARRAY	CONTENTS OF ELEMENT
7	shelf[0]	7
23	shelf[1]	23
34	shelf[2]	34
0	shelf[3]	0
0	shelf[4]	0
12	shelf[5]	12
9	shelf[6]	9
...	...	...
...	...	...
23	shelf [99]	23

In this case, the contents of a variable (an element of the array) represents a number of products the shelf contains. Elements are accessed by indexing, i.e. by specifying their ordinal number (index):

```
1 shelf[4] = 12;    // 12 items is 'placed' on shelf [4]
2 temp = shelf [1]; // Variable shelf[1] is copied to
3                // variable temp
```

Elements can be assigned contents during array declaration. In the following example, the array named calendar is declared and each element is assigned specific number of days:

```
1 unsigned char calendar [12] = {31,28,31,30,31,30,31,31,30,31,30,31};
```

TWO-DIMENSIONAL ARRAY

Apart from one-dimensional arrays which could be thought of as a list, there are also multidimensional arrays in C language. In a few following sentences we are going to describe only two-dimensional arrays called *matrices* which can be thought of as tables. A twodimensional array is declared by specifying data type of the array, the array name and the size of each dimension. Look at the example below:

```
1 component_type array_name [number_of_rows] [number_of_columns];
```

number_of_rows and *number_of_columns* represent the number of rows and columns of a table, respectively.

```
1 int Table [3][4]; // Table is defined to have 3 rows and 4 columns
```

This array can be represented in the form of a table.

table[0][0]	table[0][1]	table[0][2]	table[0][3]
table[1][0]	table[1][1]	table[1][2]	table[1][3]
table[2][0]	table[2][1]	table[2][2]	table[2][3]

Similar to vectors, the elements of a matrix can be assigned values during array declaration. In the following example, the elements of the two-dimensional array **Table** are assigned values. As seen, this array has two rows and three columns:

```
1 int Table[2][3] = {{3,42,1}, {7,7,19}};
```

The matrix above can also be represented in the form of a table the elements of which have the following values:

3	42	1
7	7	19

FUNCTIONS

Every program written in C language consists of larger or smaller number of functions. The main idea is to divide a program into several parts using these functions in order to solve the actual problem easier. Besides, functions enable us to use the skills and knowledge of other programmers. For example, if it is necessary to send a string to an LCD display, it is much easier to use already written part of the program than to start over.

Functions consist of commands specifying what should be done upon variables. They can be compared to subroutines. As a rule, it is much better to have a program consisting of large number of simple functions than of a few large functions. A function body usually consists of several commands being executed by the order they are specified.

Every function must be properly declared so as to be properly interpreted during the process of compilation. Declaration contains the following elements:

- Function name

- Function body
- List of parameters
- Declaration of parameters
- Type of function result

This is how a function looks like:

```
1 type_of_result function_name (type argument1, type argument2,...)
2 {
3     Command;
4     Command;
5     ...
6 }
```

Example:

```
1 /* Function computes the result of division of the numerator number by the denominator
2    denom. The function returns a structure of type div_t. */
3
4 div_t div(int number, int denom);
```

Note that a function does not need to have parameters, but must have brackets to be used for entering them. Otherwise, the compiler would misinterpret the function.

If the function, after being executed, returns no result to the main program or to the function it is called by, the program proceeds with execution after encountering a closing curly bracket. Such functions are used when it is necessary to change the state of the microcontroller output pins, during data transfer via serial communication, when writing data on an LCD display etc. The compiler recognizes those functions by the type of their result specified to be **void**.

```
1 void function_name (type argument1, type argument2,...)
2 {
3     Commands;
4 }
```

Example:

```
1 void interrupt() {
2     cnt++; // Interrupt causes cnt to be incremented by 1
3     PIR1.TMR1IF = 0; // Reset bit TMR1IF
4 }
```

The function can be assigned an arbitrary name. The only exception is the name `main` which has a special purpose. Namely, the program always starts execution with this function. It means that every program written in C language must contain one function named 'main' which does not have to be placed at the beginning of the program.

If it is necessary that called function returns results after being executed, the return command, which can be followed by any expression, is used:

```
1 type_of_result function_name (type argument1, type argument2,...)
2 {
3     Commands;
4     ...
5     return expression;
6 }
```

If the function contains the **return** command without being followed by *expression*, the function stops its execution when encounters this command and the program proceeds with execution from the first command following a closing curly bracket.

DECLARATION OF A NEW FUNCTION

Apart from the functions that C language 'automatically' recognizes, there are also completely new functions being often used in programs. Each 'non-standard' function should be declared at the beginning of the program.

The function declaration is called a prototype and looks as follows:

```

1 type_of_result function_name (formal parameters)
2 {
3     description of formal parameters
4     definition and declaration
5     operators
6     ...
7 }

```

Type of functions which do not return a value is **void**. If the type of result is not specifically declared in the program, it is considered to be of type **int** (signed integer). Parameters written in the function prototype define what is to be done with real parameters. Prototype function parameters are called *FORMAL PARAMETERS*. The following example declares a function which computes the volume of a cylinder.

Example:

```

1 const double PI = 3.14159; // Declare constant PI
2
3 float volume (float r, float h) // Declare type float for
4 {                               // formal parameters r and h
5     float v;                   // Declare type of result v
6     v = PI*r*r*h;             // Declare function volume
7     return v;
8 }

```

If such calculation needs to be performed later in the program (it can be the volume of a tank in practice), it is sufficient to define *REAL PARAMETERS* and call the function. During the process of compiling, the compiler is to replace formal parameters by real as shown below:

```

1 float radius=5, height=10, tank; // declare type float for
2 ...                               // real parameters radius,
3 ...                               // height and tank
4 tank = volume (radius,height);    // calculate the volume of tank
5 ...                               // by calling the volume function

```

FUNCTION LIBRARIES

Names of all functions being used in C language are stored in the file called header. These functions are, depending on their purpose, sorted in smaller files called libraries. Prior to using any of them in the program, it is necessary to specify the appropriate header file using the **#include** command at the beginning of the program. If the compiler encounters an unknown function during program execution, it will first look for its declaration in the specified libraries.

STANDARD ANSI C LIBRARIES

The functions of C language were not standardized in the beginning and software manufacturers modified them according to their needs. But C language became very popular soon and it was difficult to keep everything under control. It was necessary to introduce a sort of standard to put things in order. The established standard is called ANSI C and contains 24 libraries with functions. These libraries are usually provided with every C compiler as the most frequent operations are performed using them.

```

1 <assert.h> <complex.h> <ctype.h>
2 <errno.h> <fcntl.h> <float.h>
3 <inttypes.h> <iso646.h> <limits.h>
4 <locale.h> <math.h> <setjmp.h>
5 <signal.h> <stdarg.h> <stdbool.h>
6 <stdint.h> <stddef.h> <stdio.h>
7 <stdlib.h> <string.h> <tgmath.h>
8 <time.h> <wchar.h> <wctype.h>

```

Everything you have read so far about programming in C language is just a theory. It is useful to know, but don't forget that this programming language is not much in connection with something concrete and tangible. You will experience many problems with accurate names of registers, their addresses, names of particular control bits and many others while writing your first program in C language. The bottom line is that it is not sufficient to be familiar with the theory of C language to make the microcontroller do something useful.



2.2 The basics of C programming language by MikroElektronika is licensed under a Creative Commons Attribution 4.0 International License, except where otherwise noted.